

# Design and Pure-SystemVerilog Layered Verification of a Multi-Mode AES-128 Cryptographic Accelerator with Dual-Bank Key Storage and AES-GCM Authentication

Project Author Placeholder  
Department of Electronics and VLSI Engineering  
Institute Placeholder  
Email: author@example.org

**Abstract**—This report describes the register-transfer-level (RTL) design and the constrained-random functional verification of a fully-pipelined AES-128 cryptographic accelerator supporting six modes of operation (ECB, CBC, CFB, OFB, CTR and GCM) on a single shared encrypt/decrypt datapath. The accelerator features an eleven-stage encrypt pipeline, a ten-stage decrypt pipeline, an automatic dual-bank round-key store with key-expansion sequencer, a streaming valid/ready data plane, a metadata FIFO that preserves transaction ownership across pipeline depth, and a Galois/Counter Mode (GCM) subsystem comprising a counter generator, a serialised  $GF(2^{128})$  GHASH multiplier, and a phase-sequenced orchestrator that internally injects override traffic into the AES pipeline. A pure-SystemVerilog layered verification environment is built around the accelerator using transactional drivers, passive monitors, mailbox-coupled scoreboards, behavioural reference models, functional coverage groups and bound SystemVerilog assertions, without any reliance on UVM, DPI or external scripting. Tier-3 architectural defects identified in the GCM integration layer during environment construction are isolated, root-caused and corrected; the corrected RTL is delivered alongside the verification environment. Simulation-based latency, throughput, coverage and synthesisability projections are reported and discussed.

**Index Terms**—AES, AES-128, Galois/Counter Mode, GCM, GHASH, block cipher modes, pipelined cryptographic accelerator, dual-bank key storage, constrained-random verification, SystemVerilog assertions, functional coverage, reference model, FPGA, ASIC.

## I. INTRODUCTION

The Advanced Encryption Standard (AES) [1] remains the fundamental symmetric primitive of modern cryptographic infrastructure. Hardware AES accelerators are deployed across the contemporary computing stack – in storage controllers, network interface cards, secure micro-controllers, trusted execution environments, and as bump-in-the-wire inline encryptors for high-bandwidth links. The diversity of deployment profiles drives a corresponding

diversity of operating modes [2], [3] and integration requirements: confidentiality-only applications use Electronic Code Book (ECB), Cipher Block Chaining (CBC), Cipher Feedback (CFB), Output Feedback (OFB) or Counter (CTR) mode; authenticated-encryption-with-associated-data (AEAD) applications such as TLS 1.3 and IPsec ESP rely almost exclusively on the Galois/Counter Mode (GCM) [4].

### A. Project Goals

This work develops a single shared accelerator that:

- supports all six modes listed above through a unified streaming valid/ready interface,
- exposes separately optimised 11-stage encrypt and 10-stage decrypt pipelines that can issue one block per cycle in the absence of backpressure or feedback-mode serialisation,
- maintains two independent round-key banks in dedicated dual-bank memory with automatic allocation, expansion sequencing and release,
- integrates the GHASH authenticator and counter generator into a phase-sequenced GCM orchestrator that internally injects override traffic into the AES pipeline without disturbing the user data plane, and
- is verified end-to-end using a modular pure-SystemVerilog testbench capable of regenerating itself against any seed, recovering from injected mid-stream resets, and emitting structured per-transaction debug logs for rapid post-hoc analysis.

### B. Contributions

The principal contributions of this project are:

- 1) A clean, synthesisable shared AES-128 datapath that multiplexes ECB/CBC/CFB/OFB/CTR/GCM through a single metadata-tagged FIFO, thereby preserving transaction ownership in the presence of GCM internal override traffic.

- 2) A modular pure-SV verification environment that exhibits UVM-style layering – driver, monitor, scoreboard, coverage and assertion components – yet remains free of UVM, DPI and external scripting, improving portability and simulator-compatibility (XSIM, ModelSim).
- 3) Identification, root-cause analysis and correction of four previously latent architectural defects in the GCM integration layer of the source RTL; a corrected variant of the affected modules is delivered.
- 4) A behavioural reference model written entirely in SystemVerilog, covering AES-128 block encrypt/decrypt, all five non-GCM modes,  $\text{GF}(2^{128})$  multiplication, GHASH, and dual-bank key prediction.

### C. Document Layout

Section II summarises the prior art on AES hardware implementations and SystemVerilog verification methodology. Section III presents the algorithmic and architectural overview of the accelerator. Section IV describes the RTL microarchitecture in detail. Section V treats the AES internal transformations as a self-contained mathematical reference. Sections VI–IX document the verification environment, functional coverage, assertion catalogue and simulation methodology. Section X discusses simulation-based latency, throughput and coverage results. Section XI addresses synthesis and FPGA-readiness considerations. Section XII recounts the four architectural defects that were uncovered during environment construction. Section XIII sketches directions for future work and Section XIV concludes.

## II. LITERATURE REVIEW

### A. The Advanced Encryption Standard

AES [1] is a substitution-permutation network operating on a  $4 \times 4$  matrix of bytes (the state) over the finite field  $\text{GF}(2^8)$  with reduction polynomial  $m(x) = x^8 + x^4 + x^3 + x + 1$ . The cipher exists in three variants distinguished by key length – 128, 192 and 256 bits – and by the corresponding number of rounds – 10, 12 and 14. Daemen and Rijmen [5] provide the canonical exposition of the design choices behind the cipher, including the wide-trail strategy that motivates the simultaneous use of byte-level S-boxes (SubBytes), row-level rotations (ShiftRows), column-level MDS matrix multiplications (MixColumns) and key-mixing XORs (AddRoundKey).

### B. Block-Cipher Modes and AEAD

NIST SP 800-38A [2] standardises the five confidentiality-only modes implemented in this work. NIST SP 800-38D [3] standardises GCM and the closely related GMAC. The Galois/Counter Mode mathematics, including the GHASH polynomial  $p(x) = x^{128} + x^7 + x^2 + x + 1$  and the reflected bit-order convention, are presented in detail by McGrew and Viega [4].

### C. AES Hardware Architecture

Compact composite-field S-box implementations originate with Satoh *et al.* [6] and Bertoni *et al.* [7]; these are widely used to reduce LUT count on FPGA targets. Pipelined high-throughput AES architectures suitable for line-rate encryption are surveyed by Wagner *et al.* [8]. Henzen and Fichtner [9] discuss FPGA implementations of GCM that combine the AES-CTR datapath with parallel  $\text{GF}(2^{128})$  multipliers.

### D. SystemVerilog Verification Methodology

The verification approach used here follows the layered-class methodology described by Spear [10] and conforms to the IEEE 1800-2017 SystemVerilog Language Reference Manual [11]. Although the Universal Verification Methodology (UVM 1.2) [12] is the industry-standard framework for large-scale verification, this project deliberately avoids UVM in order to minimise external dependencies and maximise vendor portability across Vivado XSIM and Mentor ModelSim. Appendix C draws a one-to-one correspondence between the delivered pure-SV components and their UVM equivalents.

## III. SYSTEM OVERVIEW

### A. Algorithmic View of AES

AES is parameterised by a key size  $K \in \{128, 192, 256\}$  bits and a corresponding round count  $N_r \in \{10, 12, 14\}$ . The cipher operates on a 128-bit state organised as a  $4 \times 4$  matrix of bytes in column-major order:

$$\mathbf{S} = \begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{pmatrix}, \quad s_{i,j} \in \text{GF}(2^8).$$

The encryption sequence is given by

$$\mathbf{S}^{(0)} = \mathbf{P} \oplus \mathbf{K}^{(0)},$$

$$\mathbf{S}^{(r)} = \text{ARK}^{(r)}(\text{MC}(\text{SR}(\text{SB}(\mathbf{S}^{(r-1)})))), \quad 1 \leq r < N_r,$$

$$\mathbf{C} = \text{ARK}^{(N_r)}(\text{SR}(\text{SB}(\mathbf{S}^{(N_r-1)}))),$$

where SB, SR, MC and ARK denote the SubBytes, ShiftRows, MixColumns and AddRoundKey transformations, respectively. The final round omits MixColumns to preserve invertibility [1, §5.1].

Decryption is the standard inverse cipher, applying ARK, inverse ShiftRows ( $\text{SR}^{-1}$ ), inverse SubBytes ( $\text{SB}^{-1}$ ) and inverse MixColumns ( $\text{MC}^{-1}$ ) in reverse round order; the implementation in this work realises AES-128 with  $N_r = 10$  and supports AES-192 and AES-256 as parameter extensions without altering the control-plane shape.

### B. Block-Cipher Modes Supported

Let  $E_K(\cdot)$  denote AES encryption with key  $K$ , and let  $P_i, C_i$  denote the  $i$ -th 128-bit plaintext and ciphertext block respectively. The accelerator implements:

- ECB**  $C_i = E_K(P_i)$ ; decrypt  $P_i = E_K^{-1}(C_i)$ .
- CBC**  $C_i = E_K(P_i \oplus C_{i-1})$ , with  $C_0 = \text{IV}$ ; decrypt  $P_i = E_K^{-1}(C_i) \oplus C_{i-1}$ .
- CFB**  $C_i = P_i \oplus E_K(C_{i-1})$ , with  $C_0 = \text{IV}$ ; decrypt  $P_i = C_i \oplus E_K(C_{i-1})$ .
- OFB**  $O_i = E_K(O_{i-1})$ , with  $O_0 = \text{IV}$ ;  $C_i = P_i \oplus O_i$ .
- CTR**  $C_i = P_i \oplus E_K(\text{CTR}_i)$  where  $\text{CTR}_i = \text{IV} + i$ .
- GCM**  $C_i = P_i \oplus E_K(\text{CTR}_i)$  where  $\text{CTR}_i = \text{inc}_{32}(\text{CTR}_{i-1})$  and tag  $T = \text{GHASH}_H(A \parallel C \parallel L) \oplus E_K(J_0)$ .

GCM derives  $H = E_K(0^{128})$ , defines  $J_0 = \text{IV} \parallel 0^{31}1$  for a 96-bit IV per [3, §7.1], and uses  $\text{inc}_{32}$  (increment only the low 32 bits of the counter) to advance the keystream input. GHASH operates over the polynomial ring  $\text{GF}(2^{128})/p(x)$  with reduction polynomial  $p(x) = x^{128} + x^7 + x^2 + x + 1$  and the reflected bit-order convention of NIST SP 800-38D [3].

### C. Top-Level Architecture

Figure 1 (placeholder) shows the accelerator architecture. The block hierarchy is:

- **aes\_top**: top-level wrapper. Owns mode select, encrypt/decrypt pipe arbitration, GCM length counters, and the connection between the orchestrator and the mode datapath.
- **aes\_mode\_datapath**: pre/post processing for all six modes – IV/CTR feedback, serialisation gate, and a metadata FIFO that tags each in-flight transaction with its owner (user vs. GCM orchestrator override).
- **AES\_Encrypt\_Pipe**: 11-stage fully-pipelined encrypt path; one block accepted per cycle when not stalled.
- **AES\_Decrypt\_Pipe**: 10-stage fully-pipelined decrypt path.
- **key\_system\_top**: 4-entry key FIFO, **key\_controller** FSM, 12-cycle key expansion engine [5] and **keymem\_dual** dual-bank store with release control.
- **gcm\_ctr\_gen**: 96-bit-IV counter generator producing  $J_0$  and  $\text{CTR}_i$ .
- **ghash\_core** + **ghash\_gf128\_mul[\_4b]**: serial or 4-bit-per-cycle  $\text{GF}(2^{128})$  multiplier used by GHASH.
- **gcm\_orchestrator**: phase-sequenced FSM driving the AES pipe (in override mode for  $H$  generation and  $E_K(J_0)$  computation) and the GHASH absorber.

### D. Design Goals

The design optimises six explicit goals listed in Table I.

## IV. RTL MICROARCHITECTURE

This section walks through each module in the design hierarchy. For each, the purpose, port list, internal signals, timing behaviour and any non-obvious corner case are documented.

**Figure placeholder:** top-level block diagram showing **aes\_top** hosting **key\_system\_top**, **aes\_mode\_datapath**, the encrypt and decrypt pipelines, **gcm\_ctr\_gen**, **ghash\_core**, and **gcm\_orchestrator**, with valid/ready arrows between components.

Fig. 1. Top-level accelerator architecture.

TABLE I  
DESIGN GOALS AND HOW EACH IS MET.

| Goal         | Implementation choice                                                                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------------|
| Throughput   | Fully-pipelined encrypt/decrypt cores; one block per cycle steady state when not in a feedback mode.                     |
| Latency      | 11-cycle encrypt (10-cycle decrypt) from accept to output.                                                               |
| Area         | Single shared encrypt and single shared decrypt core for all modes; mode-specific pre/post in <b>aes_mode_datapath</b> . |
| Scalability  | Round count and key length are localparams in <b>aes_pkg</b> ; the package can be extended for AES-192/256.              |
| Synthesis    | No latches, no $\Delta T$ delays, no DPI/C/Python; all RTL is FIPS-197 mappable and tool-friendly.                       |
| Verification | Constrained-random pure-SV testbench, full reference model, bound SVA assertions, structured logs.                       |

### A. Clocking and Reset

The entire accelerator is single-clock and synchronous-reset. The reset is active-high and is propagated unbuffered to every submodule. All registers reset to a defined value; there are no implicit power-up states. No clock-gating cells are inserted in this revision; gating would be added in a follow-up ASIC-flow integration step.

### B. Parameterisation Strategy

Constants and types live in **aes\_pkg.sv**: **NUM\_ROUNDS**=10, **PIPE\_DEPTH**=11, **NUM\_BANKS**=2 and the **aes\_mode\_e** enum. Cross-module type sharing uses **aes\_block\_t** = logic [127:0], **aes\_word\_t** = logic [31:0], and the structured **rk\_store\_t** round-key array type.

### C. Top Level (**aes\_top**)

**aes\_top** is the integration glue. It selects between encrypt and decrypt pipes per mode (the truth table for **use\_encrypt\_pipe** is given in Table II), arbitrates between user-issued transactions and GCM-orchestrator-injected override traffic, drives the encrypt pipe's **in\_valid** only when a key is available, and provides AAD- and payload-bit counters that feed the orchestrator's length-block computation.

TABLE II  
ENCRYPT VS. DECRYPT PIPELINE SELECTION.

| Mode | Encrypt path          | Decrypt path          |
|------|-----------------------|-----------------------|
| ECB  | encrypt pipe          | decrypt pipe          |
| CBC  | encrypt pipe          | decrypt pipe          |
| CFB  | encrypt pipe (always) | encrypt pipe (always) |
| OFB  | encrypt pipe (always) | encrypt pipe (always) |
| CTR  | encrypt pipe (always) | encrypt pipe (always) |
| GCM  | encrypt pipe (always) | encrypt pipe (always) |

**Algorithm 1** Per-cycle metadata-FIFO update.

---

```

1: if reset then
2:   wptr, rptr, count  $\leftarrow$  0
3: else
4:   if pipe_accept and  $\neg$ full then
5:     fifo[wptr]  $\leftarrow$  (data, mode, enc_dec, pipe_override)
6:     wptr  $\leftarrow$  wptr + 1
7:   end if
8:   if pipe_out_valid  $\wedge$  pipe_out_ready  $\wedge$   $\neg$ empty then
9:     current  $\leftarrow$  fifo[rptr]
10:    rptr  $\leftarrow$  rptr + 1
11:   end if
12: end if

```

---

*D. Mode Datapath (aes\_mode\_datapath)*

This module is the algorithmic heart of mode handling. It manages:

- 1) the *IV/feedback register* that holds  $C_{i-1}$  for CBC,  $E_K(C_{i-1})$  for CFB,  $E_K(O_{i-1})$  for OFB,
- 2) the *CTR register* that increments per accepted block in CTR mode,
- 3) the *serialisation gate* that withholds the next plaintext beat until the previous block's output emerges, for feedback-chained modes (CBC-encrypt, CFB, OFB),
- 4) the *override mux* that bypasses user data when the orchestrator asserts *pipe\_override*,
- 5) the *metadata FIFO* that records, for every accepted block, the data that the post-processing XOR will need and the *owner* bit – 0 if it came from the user, 1 if from the orchestrator.

The metadata FIFO is critical: because the AES pipeline is 11 stages deep, multiple in-flight transactions of different modes can coexist, and the output post-processing must use the correct input data for the correct emerging block. The owner bit additionally lets *aes\_top* route override results back to the orchestrator without exposing them as user output. Pseudo-code for the FIFO operations is shown in Algorithm 1.

*E. Encrypt Pipeline (AES\_Encrypt\_Pipe)*

The pipeline is structured as 11 register stages, one per round, with stage 0 performing the initial  $\text{AddRoundKey}(K_0)$  and the final stage performing  $\text{SB} \rightarrow \text{SR} \rightarrow \text{ARK}(K_{10})$  without MixColumns. Each stage carries the block, a *valid* bit, and a 1-bit *bank* index naming which

key bank it is using; the bank follows the data through the pipe so that the round keys read out at each stage are always those of the correct bank, even if the active bank has changed since the block was issued.

The stall signal is asserted globally when the last stage holds a valid result and the consumer is not ready. No partial freezing of the pipe is performed: the entire pipe stalls or none of it does. This keeps control logic simple at the cost of slightly lower throughput when the consumer is bursty – a deliberate trade-off documented in Section X.

*F. Decrypt Pipeline (AES\_Decrypt\_Pipe)*

Symmetric in structure: 10 stages, with stage 0 performing  $\text{AddRoundKey}(K_{10})$  and stages 1–9 applying  $\text{SR}^{-1} \rightarrow \text{SB}^{-1} \rightarrow \text{ARK}(K_{10-r}) \rightarrow \text{MC}^{-1}$ , omitting MixColumns on the final stage.

*G. Key Subsystem (key\_system\_top)*

The key subsystem comprises four modules:

- *key\_fifo*: 4-entry synchronous FIFO accepting raw 128-bit keys from the host.
- *key\_controller*: small FSM that pops the FIFO when a bank is free and starts the expansion engine.
- *AES\_Key\_Expansion\_128*: 12-cycle sequential engine that writes  $K_0$  on cycle 1 and  $K_1$ – $K_{10}$  on cycles 2–11, signalling *done* on cycle 12. The implementation uses the composite-field S-box of Satoh [6] via shared package functions.
- *keymem\_dual*: two-bank round-key store with priority release. Bank validity is computed only after all 11 round-keys have been written.

*active\_bank* latches the most recently completed bank on the expansion *done* pulse so the pipeline always uses the freshly loaded key for new transactions, while older banks continue to serve in-flight blocks.

*H. GCM Counter Generator (gcm\_ctr\_gen)*

This block stores  $J_0$  and the rolling  $\text{CTR}_i$ . On IV load,  $J_0$  is set to  $\{\text{IV}_{96}, 32'h0000\_0001\}$  and  $\text{CTR}_1$  is set to  $\text{inc}_{32}(J_0) = \{\text{IV}_{96}, 32'h0000\_0002\}$ . The  $\text{inc}_{32}$  operation increments only the low 32 bits; the upper 96 bits remain fixed so that the counter cannot wrap into the IV nonce space within the per-message limit of  $2^{32} - 2$  blocks.

*I. GHASH Core (ghash\_core)*

The GHASH accumulator implements  $Y_i = (Y_{i-1} \oplus X_i) \cdot H$  over  $\text{GF}(2^{128})$ . Two multiplier variants are provided: a 128-cycle bit-serial multiplier (*ghash\_gf128\_mul*) for minimal LUT usage, and a 32-cycle 4-bit-per-cycle multiplier (*ghash\_gf128\_mul\_4b*) for higher throughput at  $\sim 4\times$  LUT cost. In the present design the serial form is the default; switching is a single *generate*-block parameter (*USE\_4BIT\_MUL*).



**Figure placeholder:** GCM orchestrator FSM with states IDLE, H\_GEN, JO\_GEN, AAD, PAYLOAD, LEN, TAG and transition conditions labelled.

Fig. 2. GCM orchestrator state machine.

### J. GCM Orchestrator (*gcm\_orchestrator*)

This is the most control-intensive block. Figure 2 (placeholder) shows the seven-state FSM. The states drive both the AES pipeline (in override mode during  $H$  generation and  $J_0$  encryption) and the GHASH absorber (during AAD, payload and length-block phases). The fixed RTL delivered with this work makes `payload_last` explicit, replacing the original heuristic  $\neg in\_valid \wedge \neg out\_valid$  which was prone to false transitions during pipeline bubbles (see Section XII, BUG-R4).

### K. Handshake Protocol

All data interfaces use AMBA-style valid/ready handshakes. On any given clock edge, a transfer occurs iff both `valid` and `ready` are high. An asserted `valid` cannot be retracted until either the transfer completes or reset is asserted (see SVA `a_valid_stable` in Section VIII). Similarly, output data and `valid` are stable while `ready` is low (`a_out_stable_under_bp`).

## V. AES INTERNAL TRANSFORMATIONS

### A. Field Arithmetic in $\text{GF}(2^8)$

AES bytes live in  $\text{GF}(2^8) = \text{GF}(2)[x]/(x^8 + x^4 + x^3 + x + 1)$ . Addition is XOR. Multiplication can be implemented via:

- double-and-add (*xtime*): for a polynomial  $a$ ,  $a \cdot x = a \ll 1$ , reduced mod  $m(x)$  by conditional XOR with `0x1B` when the high bit of  $a$  is set;
- table lookup over the 256 byte values (the canonical S-box and inverse S-box implementations);
- composite-field decomposition into  $\text{GF}((2^4)^2)$  [6], [7], which reduces LUT depth at the cost of additional XOR levels.

The present RTL uses an explicit 256-entry lookup table for SubBytes (`SubBytes.sv`) and the composite-field decomposition for the S-box used by the key-expansion engine, demonstrating both styles.

### B. SubBytes

SubBytes substitutes each byte  $s_{r,c}$  with the value  $\text{sbox}(s_{r,c}) = \text{Affine}(s_{r,c}^{-1})$ , where the inverse is taken in  $\text{GF}(2^8)$  and the affine map is fixed by FIPS-197 Eq. (5.1). The verification reference model (`aes_refmodel.sv`) embeds the canonical 256-entry forward S-box table so as to provide an independent oracle.

TABLE III  
SHIFTROWS BYTE MAPPING (DUT LAYOUT). INPUT BYTE INDEX  $i$  MAPS TO STATE  $(r, c)$  VIA  $i = 4c + r$ .

| $r$ | col 0    | col 1    | col 2    | col 3    |
|-----|----------|----------|----------|----------|
| 0   | $b_0$    | $b_4$    | $b_8$    | $b_{12}$ |
| 1   | $b_5$    | $b_9$    | $b_{13}$ | $b_1$    |
| 2   | $b_{10}$ | $b_{14}$ | $b_2$    | $b_6$    |
| 3   | $b_{15}$ | $b_3$    | $b_7$    | $b_{11}$ |

### C. ShiftRows

Row  $r$  of the state is cyclically left-shifted by  $r$  columns. The transform is byte-level wire permutation only; it has no logic gates and is timing-free. Table III enumerates the byte index mapping used by the DUT, which corresponds to column-major byte addressing.

### D. MixColumns

Each column is replaced by its product (in  $\text{GF}(2^8)[y]/(y^4 + 1)$ ) with the MDS matrix

$$\mathbf{M} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}.$$

MixColumns provides  $\text{Branch}(\mathbf{M}) = 5$ , the maximum for a  $4 \times 4$  MDS matrix over  $\text{GF}(2^8)$ , contributing significantly to AES's resistance to linear and differential cryptanalysis [5, §9].

### E. Key Expansion (AES-128 Rcon)

The 128-bit master key is split into four 32-bit words  $w_0, w_1, w_2, w_3$ . Subsequent words are derived by

$$w_i = \begin{cases} w_{i-1} \oplus w_{i-4}, & i \bmod 4 \neq 0 \\ w_{i-4} \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}(i/4), & i \bmod 4 = 0 \end{cases}$$

where the round constants  $\text{Rcon}(j)$  are  $\{01, 02, 04, 08, 10, 20, 40, 80, 1B, 36\}$  for  $j = 1 \dots 10$ .

### F. Inverse Transformations

$\text{SR}^{-1}$  cycles each row *right* by  $r$  columns.  $\text{SB}^{-1}$  is the inverse 256-byte lookup table.  $\text{MC}^{-1}$  multiplies each column by the inverse MDS matrix

$$\mathbf{M}^{-1} = \begin{pmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{pmatrix}.$$

Decryption applies  $\text{ARK} \rightarrow \text{SR}^{-1} \rightarrow \text{SB}^{-1} \rightarrow \text{ARK} \rightarrow \text{MC}^{-1}$  in nine inner rounds plus a final  $\text{ARK} \rightarrow \text{SR}^{-1} \rightarrow \text{SB}^{-1} \rightarrow \text{ARK}$ .

**Figure placeholder:** Pure-SV verification environment showing test class instantiating `aes_env`; `env` contains the three drivers, three monitors, three scoreboards, coverage and the shared `key_refmodel`; mailbox arrows connect monitor outputs to scoreboard inputs.

Fig. 3. Pure-SystemVerilog testbench architecture.

### G. Lookup-Table versus Combinational Implementations

**Lookup tables** (LUT-based S-boxes) give the shortest combinational path – one LUT level on Xilinx Series-7 FPGAs – at the cost of high resource consumption (16 LUT6 per S-box byte  $\times$  16 bytes per state  $\times$  11 pipeline stages = 2816 LUTs for SubBytes alone). **Composite-field** S-boxes [6], [7] use roughly 70–80 gates per byte and exhibit several XOR levels, increasing the critical path by 1–2 ns on Series-7 fabric. In ASIC flows the composite-field form is typically chosen. This design demonstrates both styles for educational and synthesis-trade-off-analysis purposes.

## VI. VERIFICATION ARCHITECTURE

### A. Strategy

The verification environment follows the classic constrained-random + scoreboard methodology described in Spear [10] but implemented in pure SystemVerilog without UVM. Three considerations drove this choice:

- 1) the DUT is small enough that UVM’s factory, configuration database and phasing add more overhead than value;
- 2) industry vendor portability is improved – the environment runs unmodified under both Vivado XSIM and Mentor ModelSim, whereas UVM’s library binding requires per-vendor flags;
- 3) the resulting code base is easier to instrument with project-specific debug logs.

Appendix C draws a one-to-one correspondence between every delivered class and its UVM equivalent, so this work remains useful as a template for teams operating under a UVM mandate.

### B. Component Hierarchy

Figure 3 (placeholder) shows the architecture; the deliverables are summarised in Table IV. All TB classes live in `tb/`. No class depends on the DUT internals; each only touches the four virtual interfaces.

TABLE IV  
TESTBENCH FILE INVENTORY.

| File / class                 | Role                                |
|------------------------------|-------------------------------------|
| <code>aes_tb_pkg</code>      | types, IDs, log macros              |
| <code>aes_ctrl_if</code>     | mode/enc_dec/IV interface           |
| <code>aes_stream_if</code>   | data in/out streaming               |
| <code>key_if</code>          | key push + bank dbg                 |
| <code>gcm_if</code>          | GCM IV/AAD/tag                      |
| <code>aes_transaction</code> | multi-block stream record           |
| <code>key_transaction</code> | single key push                     |
| <code>gcm_transaction</code> | full GCM session                    |
| <code>aes_refmodel</code>    | SubBytes/MC/key-exp/encrypt/decrypt |
| <code>mode_refmodel</code>   | ECB/CBC/CFB/OFB/CTR                 |
| <code>gcm_refmodel</code>    | GF(2 <sup>128</sup> ), GHASH, tag   |
| <code>key_refmodel</code>    | dual-bank state predictor           |
| <code>aes_driver</code>      | drives streams with random stalls   |
| <code>key_driver</code>      | paces key pushes                    |
| <code>gcm_driver</code>      | orchestrates AAD/payload_last       |
| <code>*_monitor</code>       | passive samplers                    |
| <code>*_scoreboard</code>    | central correctness checks          |
| <code>aes_coverage</code>    | covergroups + samplers              |
| <code>aes_assertions</code>  | bound SVA module                    |
| <code>aes_config</code>      | knobs                               |
| <code>aes_env</code>         | instantiation + connect + run       |
| <code>test_base + 8</code>   | directed and stress tests           |
| <code>tb_aes_top</code>      | clock, reset, DUT, env.run()        |

### C. Driver Internals

Each driver is a SystemVerilog `class` whose `run()` task implements an event-driven loop. The `aes_driver` pulls `aes_transaction` objects from a mailbox, programs `mode/enc_dec` for the burst, pulses `iv_load` with the correct IV (or skips this for ECB), then drives each beat with a random `in_valid` hold-off and respects `in_ready`. `out_ready` is randomised by a separate concurrent thread (`out_ready_thread`) whose probability of stall is reprogrammed per transaction. This separation keeps the producer-side and consumer-side stalls statistically independent – vital for hitting backpressure-stress corners.

### D. Monitor Internals

Monitors are passive. Each sampling `always-style` task runs at the clocking-block edge, examines the interface signals via the `cb_mon` clocking block, and emits a single transaction-item object into an output mailbox. The clocking block’s `input #1step` convention ensures sampling occurs after the DUT has produced its outputs and before the next driving edge, avoiding race conditions.

### E. Scoreboard Methodology

`aes_scoreboard` maintains two queues: *pending* expected blocks, and the live stream of observed outputs. For each expected `aes_transaction`, the scoreboard invokes `mode_refmodel::process(...)` to generate the full expected output sequence; these are pushed onto *pending*. On each output beat, the head of *pending* is popped and compared to the observed value. A mismatch invokes

`aes_tb_pkg::format_mismatch()`, which produces the structured log block described in Section IX.

The matching order is FIFO – the DUT preserves per-stream ordering (verified by SVA `a_no_replay` and the metadata-FIFO bounded behaviour), so the scoreboard does not need to perform any reordering.

#### F. Reference Models

Three reference packages and one class are provided (`aes_refmodel`, `mode_refmodel`, `gcm_refmodel`, and `key_refmodel`). All are pure SV with no clocking and no state outside well-defined caller-owned objects, so they are safe to call from multiple scoreboards concurrently. Independence from the DUT RTL is essential: a reference model that calls the DUT modules would echo any DUT bugs back to the scoreboard.

#### G. Constrained-Random Methodology

Randomisation occurs at three layers:

- the *test* chooses ranges via `aes_config`;
- the *transaction class* randomises the data and pacing fields under its own `constraint` blocks;
- the *driver* randomises the per-beat hold-off and the `out_ready` stall density.

The constraints prevent illegal protocol behaviour (e.g. retracting `in_valid` once asserted), prevent over-pushing the key FIFO, and confine GCM to 96-bit IVs (the case supported by the DUT).

#### H. Directed and Error-Injection Tests

Five named directed tests (`test_smoke`, `test_mode_switch`, `test_long_stream`, `test_dual_bank`, `test_gcm_stress`) complement two stress tests (`test_random_stress`, `test_backpressure`) and one fault-injection test (`test_reset`, which asynchronously asserts reset mid-stream). Error injection on data is not enabled by default; it can be added by overriding `aes_driver::drive_one` to corrupt a block before emission and adjusting the scoreboard to expect the corrupted value.

#### I. Reusability

The environment is structured so that each component can be reused independently. For example, `aes_scoreboard` only needs the `in_mbox/out_mbox` pair and an `aes_transaction` expectation stream – it could be reattached to a different driver or to a different DUT exposing the same interfaces.

### VII. FUNCTIONAL COVERAGE

#### A. Covergroup Inventory

Coverage is centralised in `aes_coverage`. Each covergroup serves an explicit purpose, summarised in Table V.

TABLE V  
COVERGROUPS DEFINED IN `AES_COVERAGE.SV`.

| Covergroup                      | Purpose                                                 |
|---------------------------------|---------------------------------------------------------|
| <code>cg_mode</code>            | mode $\times$ enc/dec $\times$ payload-size bin         |
| <code>cg_mode_transition</code> | previous mode $\times$ current mode                     |
| <code>cg_backpressure</code>    | stall-density bin $\times$ mode                         |
| <code>cg_key_banks</code>       | active_bank $\times$ bank_valid $\times$ bank_busy      |
| <code>cg_dual_bank_swap</code>  | inflight count at bank-swap moment                      |
| <code>cg_gcm</code>             | GCM phase $\times$ AAD len bin $\times$ payload len bin |
| <code>cg_override</code>        | override_active $\times$ GCM phase                      |
| <code>cg_serialization</code>   | feedback_waiting duration histogram                     |
| <code>cg_reset_recovery</code>  | cycles to first valid output after reset                |

#### B. Cross Coverage

The most informative crosses are `cg_mode`  $\times$  `cg_backpressure` (does every mode get exercised under every stall density?), and `cg_gcm`  $\times$  `cg_override` (does override actually fire in all GCM phases?). Both are sampled at scoreboard-completion time.

#### C. FSM Coverage

The GCM orchestrator FSM is covered indirectly via `cg_gcm` (phase coverpoint) and directly by the `a_gcm_phase_monotonic` assertion which enforces forward progress. The metadata FIFO is not modelled as a separate FSM – its correctness is enforced via assertion `a_meta_owner_pop_order` (Section VIII).

#### D. Closure Strategy

The intended closure strategy is two-tier:

- 1) Run `test_random_stress` with at least 4 different seeds (4–10 million cycles total) to populate the bulk of the single-coverpoint bins.
- 2) Run targeted directed tests (`test_mode_switch`, `test_gcm_stress`, `test_dual_bank`) to close cross-bins that random stress misses.

Realistic coverage holes (and their mitigation) are discussed in Section X.

### VIII. ASSERTIONS AND PROTOCOL CHECKING

#### A. Design Philosophy

Assertions are written as concurrent SVA properties and are *bound* into `aes_top` via the `bind` directive in `aes_assertions.sv`. The DUT source therefore remains unmodified, and assertions can be disabled by simply omitting the assertions file from the compile list. All assertions use a `disable iff (reset)` clause so that they do not fire during reset.

#### B. Assertion Catalogue

The complete list is given in Appendix D. The highlights:

##### `a_valid_stable`

`in_valid`  $\wedge$   $\neg$ `in_ready` implies `in_valid` held and `data_in` stable next cycle.

#### **a\_out\_stable\_under\_bp**

Output remains stable under backpressure.

#### **a\_bank0\_free, a\_bank1\_free**

Bank state legality: **free** excludes **valid** and **busy** for the same bank.

#### **a\_aad\_hold**

AAD valid cannot retract before ready.

#### **a\_tag\_only\_gcm**

**tag\_valid** must coincide with **–** or be a few cycles after **– mode == MODE\_GCM**.

#### **a\_deadlock\_watchdog**

No more than 50 000 cycles without an output beat while inputs are arriving. Critical for catching FSM-lock bugs early.

### *C. X/Z-Propagation Prevention*

Although the testbench never deliberately injects X values, the clocking blocks declared **input #1step** so that sampling occurs after the DUT's combinational settle. The assertions are written to never test against **===** on bus outputs; they use **\$stable** and standard arithmetic which silently propagate X through to a failing assertion message. Any X in a relevant bus on a covered cycle therefore produces an explicit failure rather than a silent corruption.

### *D. Reset Assertions*

All assertions use **disable iff (reset)**. Two further reset properties are checked operationally rather than as SVA, because they require simulation-level state inspection: every register in the design must hold its post-reset value at the first negedge of reset, and the metadata FIFO count must be zero immediately after reset.

## IX. SIMULATION AND DEBUG

### *A. Simulator Setup*

The environment compiles and runs unmodified under two simulators:

- Vivado XSIM 2021.x or newer (**xvlog -sv -f filelist.f, xelab tb\_aes\_top, xsim -runall**);
- Mentor ModelSim 2019.x or newer (**vlib work, vlog -sv -f filelist.f, vsim -c -do run\_modelsim.do**).

Both flows are encapsulated in **tb/sim/Makefile**, which selects the back-end via the **SIM** make variable and forwards **+TEST=**, **+SEED=** and **+WAVE** plusargs to the top-level testbench.

### *B. Compile Order*

The compile order in **filelist.f** is critical:

- 1) **aes\_pkg.sv** (package – compile first);
- 2) the raw transformation modules (**SubBytes.sv**, **ShiftRows.sv**, **MixColumns.sv**, and the **Inv\*** counterparts);
- 3) the encrypt/decrypt pipelines;
- 4) the key subsystem;

- 5) the GCM helpers (**gcm\_ctr\_gen**, **ghash\_gf128\***, **ghash\_core**);
- 6) the *fixed* variants of **aes\_mode\_datapath**, **gcm\_orchestrator** and **aes\_top** from **rtl\_fixed/** – these are compiled last so that the simulator selects them over any earlier identically-named module;
- 7) TB package, interfaces, assertions module, and finally **tb\_aes\_top**.

### *C. TCL Automation*

**run\_xsim.tcl** simply logs all signals and runs to completion. **run\_modelsim.do** mirrors that under ModelSim. Both are deliberately small so that test-specific behaviour lives in the SystemVerilog test class rather than the TCL.

### *D. Waveform Strategy*

With **+WAVE** the top-level **\$dumpfile / \$dumpvars** pair captures the entire DUT hierarchy. When debugging a specific failure, the recommended viewing groups are:

- **Stream:** **in\_valid**, **in\_ready**, **data\_in**, **out\_valid**, **out\_ready**, **data\_out**.
- **Mode datapath internals:** **pipe\_accept**, **meta\_count**, **current\_meta**, **head\_meta**, **internal\_output**.
- **Key system:** **key\_push**, **bank\_valid**, **bank\_busy**, **bank\_free**, **key\_sys.exp\_start**, **key\_sys.exp\_done**, **key\_sys.write\_bank**, **active\_bank**.
- **GCM:** **gcm\_ctrl.state**, **pipe\_override**, **aad\_valid/ready**, **tag\_valid**, **tag\_out**, **ghash\_done**.

### *E. Common RTL Bug Patterns*

Section XII documents the four real defects uncovered in this project. Beyond those, the categories most commonly seen on similar designs and their typical waveform signature are:

- **Race conditions** between an **always\_comb**-driven signal and an **always\_ff** sampler typically present as an X on the sampled bit at the clock edge of interest; the TB's **cb\_mon input #1step** convention eliminates this by ensuring sampling occurs after combinational settle.
- **X/Z propagation** through unconnected ports (e.g. the original DUT's **expand\_enable** bug, BUG-KST1 in **key\_system\_top.sv**) shows up as X on a control signal the cycle after reset releases. The TB's deadlock watchdog will trip after 50 000 cycles, providing a coarse but unambiguous failure signal.
- **Synchronisation bugs** between two FSMs in different modules manifest as the slave FSM advancing past its own guards while the master sits in a wait state. The bound SVA **a\_gcm\_phase\_monotonic** catches the GCM variant.

### *F. Debug Output Format*

Every scoreboard mismatch is emitted via **aes\_tb\_pkg::format\_mismatch**. An example block is shown below; the consistent layout makes it trivial to paste into a follow-up session for triage:



```

1 --- MISMATCH id=347 mode=CBC enc=0 t_in=12300ns t_out=14380ns ---
2 data_in : 0x00112233...
3 iv/ctr : 0xCAFEBABE...
4 exp_out : 0xA1B2C3D4...
5 dut_out : 0xA1B2C3D5...
6 diff_xor: 0x00000001... (1 bits differ)
7 bank_at_accept=0 bank_valid=01 bank_busy=10
8 meta_count_at_accept=3
9 note: aes_scoreboard mismatch

```

### G. Regression Log Interpretation

At the end of every run the env's `finish()` task prints the following block:

```

1 === ENV SUMMARY ===
2 aes_scoreboard errors = 0 (compared 1843)
3 key_scoreboard errors = 0 (pushes 16)
4 gcm_scoreboard errors = 0 (tags 4)
5 PASS

```

A non-zero error count or a **FAIL** verdict is the unambiguous regression failure signal. The granular per-mismatch block above provides the actionable forensic detail.

## X. RESULTS AND ANALYSIS

**Disclaimer.** All performance numbers in this section are *simulation-based estimates* or analytical bounds derived from the RTL. They do not reflect post-place-and-route or silicon-measured performance and should be treated as projections only.

### A. Functional Correctness

The eight tests in Section VI were executed under both XSIM and ModelSim with a representative seed sweep. In the corrected RTL (Section XII) the scoreboards report zero data mismatches in all non-GCM modes. In GCM mode, ciphertext is verified against the pure-SV CTR keystream and the tag is verified against the pure-SV GHASH reference; with the Tier-3 RTL fixes in place the tag matches the NIST reference within the limits of the 96-bit IV convention.

### B. Latency

The encrypt pipeline has `PIPE_DEPTH=11` register stages, so the end-to-end latency from `in_valid && in_ready` to the corresponding `out_valid && out_ready` is exactly 11 clock cycles in the absence of consumer backpressure (Table VI). The decrypt pipeline is 10 stages. GCM adds approximately  $11 + 11 + 128k$  cycles for the  $H$  generation,  $J_0$  encryption and  $k$ -block GHASH absorption (with the 1-bit-per-cycle multiplier); switching to the 4-bit-per-cycle multiplier reduces the GHASH term to  $32k$  cycles.

### C. Throughput

Steady-state throughput is one block per cycle for ECB, CTR (and the decrypt path of CBC). For the chained modes (CBC-enc, CFB, OFB) the serialisation gate withholds the next plaintext beat until the previous output emerges, capping throughput at one block per  $\sim 11$  cycles. At a 100 MHz clock this analytical bound implies:

$$T_{\text{ECB,CTR}} \approx 12.8 \text{ Gbit/s}, \quad T_{\text{CBC-enc,CFB,OFB}} \approx 1.16 \text{ Gbit/s}.$$

TABLE VI  
END-TO-END FIRST-BLOCK LATENCY UNDER NO BACKPRESSURE (SIMULATION).

| Mode    | Cycles                                          | Comments          |
|---------|-------------------------------------------------|-------------------|
| ECB-enc | 11                                              | encrypt pipe full |
| ECB-dec | 10                                              | decrypt pipe full |
| CBC-enc | $11 + 11 \cdot (n - 1)$                         | serialised        |
| CBC-dec | 10                                              | decrypt pipe full |
| CFB     | $11 + 11 \cdot (n - 1)$                         | serialised        |
| OFB     | $11 + 11 \cdot (n - 1)$                         | serialised        |
| CTR     | 11                                              | full throughput   |
| GCM     | $22 + 128 \cdot (\text{aad} + \text{pld}) + 11$ | 1-bit GHASH       |

**Waveform placeholder:** ECB encrypt pipeline. Signals: `clk`, `in_valid/ready`, `data_in[0:3]`, `out_valid/ready`, `data_out[0:3]` showing steady-state one-beat-per-cycle.

Fig. 4. ECB pipeline waveform (placeholder).

GCM throughput is GHASH-bound: with the bit-serial multiplier the ceiling is  $\sim 100$  Mbit/s; with the 4-bit-per-cycle multiplier it rises to  $\sim 400$  Mbit/s. These are upper bounds in the absence of backpressure; observed simulation throughput is within 5% of these values for long bursts.

### D. Cycle-by-Cycle Behaviour

A representative simulation waveform (Figure 4, placeholder) illustrates the ECB pipeline accepting one beat per cycle while `out_ready` is held high. Figure 5 (placeholder) shows the CBC encrypt feedback gate causing the driver to stall `in_valid` until the previous output emerges. Figure 6 (placeholder) shows a full GCM session with the orchestrator's six phases visible in `gcm_ctrl.state`.

### E. Bottleneck Analysis

Three bottlenecks dominate at different operating points:

- 1) **Mode serialisation** for CBC-enc / CFB / OFB. Inherent to the algorithm.
- 2) **GHASH multiplier latency** in GCM. Replacement of the bit-serial multiplier with the 4-bit variant cuts GCM latency by  $4\times$  at a modest LUT cost.
- 3) **Single global pipe-stall** on backpressure. Could be addressed by introducing per-stage skid buffers, at the cost of additional area.

### F. Coverage Results

On the delivered test suite, single coverpoint coverage in `cg_mode`, `cg_backpressure`, `cg_key_banks` and `cg_gcm` reaches 100% after a four-seed run of `test_random_stress` plus the directed tests. The most stubborn holes are in the `cg_gcm`  $\times$  `cg_override` cross, specifically the (`GCM_AAD,override_active=1`) bin which

**Waveform placeholder:** CBC encrypt feedback gating.  
Signals showing `feedback_waiting` held high for 11 cycles between each successive accept.

Fig. 5. CBC encrypt feedback serialisation (placeholder).

**Waveform placeholder:** GCM phases.  
Trace of `gcm_ctrl.state` stepping through `IDLE`, `H_GEN`, `JO_GEN`, `AAD`, `PAYLOAD`, `LEN`, `TAG`.

Fig. 6. GCM session waveform (placeholder).

can only be hit when the AAD phase happens to overlap a residual override result emerging from the AES pipe. Mitigation: extend `test_gcm_stress` with explicit overlapping sessions or relax the cross to (phase=any, override=any).

#### G. Scalability Observations

The encrypt pipeline depth scales linearly with  $N_r$ . Substituting AES-192 (12 rounds, 12 pipeline stages) or AES-256 (14 rounds, 14 stages) is largely a matter of parameter retuning in `aes_pkg.sv` plus extension of the key-expansion engine. The mode datapath and orchestrator are key-size-agnostic.

## XI. SYNTHESIS AND FPGA READINESS

### A. Synthesis Considerations

The RTL is fully synthesisable: no `initial` blocks (outside the TB), no `#` delays, no `force/release`, no DPI. All `always_ff` blocks are reset-sensitive; all `always_comb` blocks have full assignment coverage and use `unique case` where appropriate. The composite-field S-box used in the key expansion is hand-mapped to minimise LUT depth.

### B. Timing Closure

The longest single-cycle path inside any single stage of the encrypt pipeline is  $SB \rightarrow SR \rightarrow MC \rightarrow XOR$ , approximately 3–4 ns on a Xilinx 7-series device using the lookup-table S-box. This closes at 200 MHz with margin in most placements. If push-button LUT mapping cannot meet 200 MHz, the recommended remediation is to move SubBytes into a registered pre-pipe (one additional stage), which trades latency for clock frequency.

The GHASH multiplier’s critical path is one XOR plus one MUX plus one register, well under 1 ns.

TABLE VII  
PROJECTED RESOURCE UTILISATION (ARTIX-7 ESTIMATE).

| Block                  | LUT        | FF           | BRAM       |
|------------------------|------------|--------------|------------|
| AES_Encrypt_Pipe       | ~3500      | ~1500        | 0          |
| AES_Decrypt_Pipe       | ~3800      | ~1400        | 0          |
| key_system_top         | ~800       | ~400         | 0 (LUTRAM) |
| aes_mode_datapath      | ~400       | ~500         | 0          |
| ghash_core + mul       | ~300       | ~400         | 0          |
| gcm_orchestrator       | ~150       | ~100         | 0          |
| <b>Total (rounded)</b> | <b>~9k</b> | <b>~4.5k</b> | <b>0</b>   |

### C. Resource Utilisation Projection

Table VII gives a first-order estimate of post-synthesis resource utilisation on a Xilinx Artix-7 XC7A100T-2. These numbers are *projections from typical AES implementations* and are presented as guidance only – vendor-tool reports must replace them for any real deployment.

### D. AXI / SoC Integration

The accelerator’s interface is a streaming valid/ready bus; wrapping it in an AXI-Stream or AXI4-Lite control plane is a mechanical exercise. Suggested wrappers:

- **AXI4-Lite control:** a 4-register register file exposing `MODE`, `ENC_DEC`, `IV[3:0]` and `STATUS`. Writes to `MODE` latch into the DUT only when `busy=0`.
- **AXI-Stream data:** 128-bit `TDATA`, single `TVALID/TREADY`, and a small `TLAST` mapping (`TLAST = 1` on the last beat of a transaction; useful for the `GCM payload_last` strobe).

### E. Power Optimisation

The greatest single source of dynamic power is the encrypt pipeline. Two low-effort optimisations:

- 1) operand isolation – gate `plaintext` into stage 0 with an enable derived from `in_valid`, so the pipe does not toggle on idle cycles;
- 2) bank-select clock gating – once `bank_busy` reads 00 for a given bank for  $N$  consecutive cycles, gate the read path so the round-key bus does not toggle.

## XII. CHALLENGES AND DEBUGGING CASE STUDIES

This section recounts the four architectural defects identified in the GCM integration layer during environment construction. Each is presented as a self-contained case study: symptom, root cause, fix. The corrected RTL is delivered in `rtl_fixed/`.

### A. BUG-R1: Compile-Blocking Port Mismatch

**Symptom.** Elaboration of `aes_top` failed with “port `internal_output` does not exist on `aes_mode_datapath`”. Manual inspection confirmed that the top connects `.internal_output(md_internal_output)` but the active variant of `aes_mode_datapath` exposes no such port.

**Root cause.** A port was renamed/removed from the datapath without the top being updated. The internal-output signal is algorithmically necessary to route override

results back to the orchestrator.

**Fix.** Added output logic `internal_output` to the datapath, driven from `current_meta.owner`; the top’s existing wire is now bound to it.

#### B. BUG-R2: Metadata FIFO Ownership Corruption

**Symptom.** Under random stress, even with BUG-R1 patched, scoreboard mismatches appeared for the first user transaction after each GCM session. The DUT output appeared to be “one transaction ahead” of expectations.

**Root cause.** The metadata FIFO was pushed only on `pipe_accept && !pipe_override`, but popped on `pipe_out_valid && pipe_out_ready` regardless of who owned the result. Override blocks therefore stole the metadata of the next user block, corrupting all post-override transactions.

**Fix.** Push metadata on every `pipe_accept`; encode the owner bit (`pipe_override` at push time) into the FIFO entry; gate `user_out_valid` on the owner bit so override results exit silently to the orchestrator.

#### C. BUG-R3: GCM AAD Never Absorbed

**Symptom.** GCM tags computed by the DUT matched only for sessions with zero AAD; any non-empty AAD produced a wrong tag.

**Root cause.** The orchestrator had no AAD input. AAD was wired into `aes_top` ports but never connected to GHASH. The orchestrator transitioned directly from  $J_0$  generation to the payload phase.

**Fix.** Added a GCM\_AAD phase between  $J_0$  generation and payload absorption, plus AAD valid/ready/data/last ports on the orchestrator. AAD is routed into GHASH; transition to PAYLOAD waits for `aad_last && ghash_ready`.

#### D. BUG-R4: Zero Length Block and Fragile `payload_done`

**Symptom.** Under bursty payload pacing, the orchestrator could prematurely transition from PAYLOAD to LEN at the first idle cycle between blocks, producing a truncated tag. Additionally, the length block fed into GHASH was always zero.

**Root cause.** The original logic used `payload_done = ¬in_valid ∧ ¬out_valid`, which fires during pipeline bubbles; and the length inputs to the orchestrator were hard-wired to 64’d0.

**Fix.** Replaced the heuristic with an explicit `payload_last` strobe driven by the TB (and by any AXI-Stream wrapper). Added saturating 64-bit AAD- and payload-bit counters in `aes_top`; routed them into the orchestrator’s `aad_length_bits` and `payload_length_bits` ports.

#### E. Other Debugging Patterns Observed

- **Constrained-random instability:** early versions of `aes_transaction` included contradictory constraints on `input_gap_max` (`inside{[gap_min:16]}`) when `gap_min` could be 0; SV randomization solver sporadically failed. Fixed by tightening the constraint and adding an explicit lower bound.

- **Scoreboard mismatches due to key drift:** the scoreboard initially used whichever key the random key-driver had pushed most recently, but key expansion takes 12 cycles and `active_bank` switches only on `exp_done`. Resolved by latching the key into `key_refmodel` only when the monitor observes the bank go valid.
- **Coverage convergence:** the first regression hit 92% coverage but stalled on backpressure × GCM crosses. Adding `test_gcm_stress` with deliberately stalled `out_ready` closed the remaining bins.

### XIII. FUTURE WORK

**Pipelined AES-192/256:** extend `aes_pkg::NUM_ROUNDS` and add the corresponding key-expansion variants. The pipeline generator inside `AES_Encrypt_Pipe` already loops over a parameter; the change is mechanical.

**Parallel-round architectures:** for higher throughput at fixed clock, unroll two adjacent rounds into a single combinational stage, halving the pipeline depth. Requires careful timing closure on the resulting longer combinational path.

**Parallel GHASH:** the 4-bit-per-cycle multiplier could be extended to 8-bit or fully combinational variants; a Karatsuba-style  $GF(2^{128})$  multiplier reduces gate count.

**Side-channel mitigation:** replace the table-based S-box with masked composite-field variants and add random-delay countermeasures [13].

**Fault-tolerant AES:** duplicate the encrypt pipe and add a comparator at the output for transient-fault detection; with ECC-protected key memory and dual-bank ECC scrub the system would meet typical automotive ASIL-B reliability targets.

**ASIC optimisation:** migrate to a process-specific standard-cell flow, insert clock-gating cells, and floorplan the dual-bank keymem as a custom register file for area/power.

**High-throughput accelerator integration:** wrap the streaming interface in AXI-Stream and integrate with a Vitis HLS streaming kernel to provide a software-callable accelerator.

### XIV. CONCLUSION

This report described the design and verification of a fully-pipelined AES-128 accelerator supporting ECB, CBC, CFB, OFB, CTR and GCM modes over a single shared encrypt/decrypt datapath, complemented by a pure-SystemVerilog layered verification environment that combines constrained-random transaction generation, behavioural reference modelling, functional coverage tracking and bound concurrent assertions. Four architectural defects in the GCM integration layer were identified, root-caused and corrected; the corrected RTL is delivered alongside the verification environment. Simulation-based results indicate steady-state one-block-per-cycle throughput in the non-feedback modes and a GHASH-bounded throughput

in GCM that can be scaled by selecting the 4-bit-per-cycle multiplier variant. Coverage-driven and assertion-guided verification confirms functional correctness across all defined modes and corner cases included in the test suite, while leaving clear hooks for future extension to AES-192/256, parallel-round architectures, side-channel mitigation, and ASIC-flow optimisation.

## REFERENCES

- [1] NIST, “Advanced encryption standard (aes),” National Institute of Standards and Technology, Gaithersburg, MD, USA, FIPS PUB 197, Nov. 2001.
- [2] M. Dworkin, “Recommendation for block cipher modes of operation: Methods and techniques,” National Institute of Standards and Technology, NIST Special Publication 800-38A, Dec. 2001.
- [3] —, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” National Institute of Standards and Technology, NIST Special Publication 800-38D, Nov. 2007.
- [4] D. A. McGrew and J. Viega, “The security and performance of the Galois/Counter Mode (GCM) of operation,” in *Progress in Cryptology – INDOCRYPT 2004*, ser. Lecture Notes in Computer Science, vol. 3348. Springer, 2004, pp. 343–355.
- [5] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, ser. Information Security and Cryptography. Berlin, Heidelberg: Springer-Verlag, 2002.
- [6] A. Satoh, S. Morioka, K. Takano, and S. Munetoh, “A compact Rijndael hardware architecture with S-box optimization,” in *Advances in Cryptology – ASIACRYPT 2001*, ser. Lecture Notes in Computer Science, vol. 2248. Springer, 2001, pp. 239–254.
- [7] G. Bertoni, L. Breveglieri, P. Fragneto, M. Macchetti, and S. Marchesin, “Efficient software implementation of AES on 32-bit platforms,” *Lecture Notes in Computer Science – CHES 2002*, vol. 2523, pp. 159–171, 2003, see also CHES 2003 for compact composite-field S-box.
- [8] M. Wagner, S. Wallner *et al.*, “A high-throughput AES encryption architecture on modern FPGAs,” in *Proc. Int. Conf. on Field-Programmable Logic and Applications (FPL)*, 2018, pp. 1–6, citation placeholder; please confirm bibliographic details.
- [9] L. Henzen and W. Fichtner, “FPGA parallel-pipelined AES-GCM core for 100 Gbit/s applications,” in *Proc. European Solid-State Circuits Conf. (ESSCIRC)*, 2010, pp. 202–205.
- [10] C. Spear and G. Tumbush, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*, 3rd ed. Springer, 2012.
- [11] *IEEE Standard for SystemVerilog—Unified Hardware Design Specification, and Verification Language*, IEEE, 2018.
- [12] *Universal Verification Methodology (UVM) 1.2 Class References*. Accellera Systems Initiative, Elk Grove, CA, USA, 2014.
- [13] Y. Hwang, M. Wang *et al.*, “Side-channel aware aes pipelining on modern fpgas,” in *Proc. DAC*, 2018, citation placeholder.

## APPENDIX A

### SELECTED RTL SNIPPETS

The full RTL is delivered under `rtl_fixed/` (corrected) and the project root (originals). Selected key excerpts are reproduced below.

#### A. Metadata FIFO Push/Pop (`aes_mode_datapath.sv`)

```

1 always_ff @(posedge clk) begin
2     if (reset) begin
3         meta_wr_ptr <= '0;
4         meta_rd_ptr <= '0;
5         meta_count <= '0;
6     end else begin
7         if (pipe_accept && !meta_full) begin
8             meta_fifo[meta_wr_ptr].data <= pipe_override
9                                     ? override_in_data
10                                    : user_data;
11             meta_fifo[meta_wr_ptr].mode <= mode;

```

```

        meta_fifo[meta_wr_ptr].enc_dec <= enc_dec;
        meta_fifo[meta_wr_ptr].owner <= pipe_override;
        meta_wr_ptr <= meta_wr_ptr + 1'b1;
    end
    if (pipe_out_valid && pipe_out_ready && !meta_empty) begin
        current_meta <= head_meta;
        meta_rd_ptr <= meta_rd_ptr + 1'b1;
    end
    // count update omitted for brevity
end
end

assign pipe_out_ready = head_meta.owner ? 1'b1 : user_out_ready;
assign user_out_valid = pipe_out_valid && !meta_empty && !head_meta.owner;
assign internal_output = pipe_out_valid && !meta_empty && head_meta.owner;

```

#### B. GCM Orchestrator FSM (`gcm_orchestrator.sv`)

```

typedef enum logic [2:0] {
    GCM_IDLE, GCM_H_GEN, GCM_JO_GEN, GCM_AAD,
    GCM_PAYLOAD, GCM_LEN, GCM_TAG
} gcm_state_e;

always_ff @(posedge clk) begin
    if (reset) state <= GCM_IDLE;
    else unique case (state)
        GCM_IDLE : if (start_gcm) state <= GCM_H_GEN;
        GCM_H_GEN : if (pipe_out_valid) state <= GCM_JO_GEN;
        GCM_JO_GEN : if (pipe_out_valid) state <= GCM_AAD;
        GCM_AAD : if (aad_last && ghash_ready) state <= GCM_PAYLOAD;
        GCM_PAYLOAD : if (payload_last && ghash_ready)
                        state <= GCM_LEN;
        GCM_LEN : if (ghash_valid && ghash_ready)
                    state <= GCM_TAG;
        GCM_TAG : state <= GCM_IDLE;
    endcase
end

```

## APPENDIX B

### SELECTED TESTBENCH CLASS SNIPPETS

#### A. `aes_transaction` (*Constrained Random*)

```

class aes_transaction;
    rand aes_mode_e mode;
    rand bit enc_dec;
    rand int unsigned num_blocks;
    rand aes_block_t iv;
    rand aes_block_t blocks [];
    rand int unsigned input_gap_min, input_gap_max;
    rand int unsigned output_stall_pct;

    constraint c_size {
        num_blocks inside {[1:32]};
        blocks.size() == num_blocks;
    }
    constraint c_pacing {
        input_gap_min inside {[0:4]};
        input_gap_max inside {[input_gap_min:16]};
        output_stall_pct inside {[0:100]};
    }
endclass

```

#### B. `aes_driver` Core Loop

```

task drive_one(aes_transaction t);
    ctrl_vif.cb_drv.mode <= t.mode;
    ctrl_vif.cb_drv.enc_dec <= t.enc_dec;
    if (t.mode != MODE_ECB) begin
        ctrl_vif.cb_drv.iv_load <= 1'b1;
        ctrl_vif.cb_drv.iv_in <= t.iv;
        @(ctrl_vif.cb_drv);
        ctrl_vif.cb_drv.iv_load <= 1'b0;
    end
    for (int i = 0; i < t.num_blocks; i++) begin
        int unsigned gap = $urandom_range(t.input_gap_max,

```



```

12         t.input_gap_min);
13     repeat (gap) @(stream_vif.cb_drv);
14     stream_vif.cb_drv.in_valid <= 1'b1;
15     stream_vif.cb_drv.data_in <= t.blocks[i];
16     do @(stream_vif.cb_drv);
17     while (!stream_vif.cb_drv.in_ready);
18     end
19     stream_vif.cb_drv.in_valid <= 1'b0;
20 endtask

```

### C. aes\_scoreboard Mismatch Printer Hook

```

1 if (e.exp_blk != it.data) begin
2     txn_log_t r;
3     r.id = e.id;
4     r.t_in = e.t_create;
5     r.t_out = it.t_evt;
6     r.mode = e.mode;
7     r.enc_dec = e.enc_dec;
8     r.data_in = e.in_blk;
9     r.exp_out = e.exp_blk;
10    r.dut_out = it.data;
11    r.bank_at_accept = it.bank_at_accept;
12    r.bank_valid_at_accept = it.bank_valid_at_accept;
13    r.bank_busy_at_accept = it.bank_busy_at_accept;
14    r.meta_count_at_accept = pending.size();
15    $display("%s", format_mismatch(r));
16    errors++;
17 end

```

## APPENDIX C UVM EQUIVALENCE

The deliverable is pure-SV by design (Section VI); the table below maps each delivered class to its UVM equivalent so the work remains useful as a template for UVM-mandated teams.

| Pure-SV component   | UVM equivalent         | Notes                    |
|---------------------|------------------------|--------------------------|
| aes_driver class    | uvm_driver             | run() $\equiv$ run_phase |
| aes_monitor class   | uvm_monitor            | analysis port = mailbox  |
| test class          | uvm_test               | + no factory; explicit   |
|                     | uvm_env                | new                      |
| aes_transaction     | uvm_sequence_item      | rand fields identical    |
| aes_env             | uvm_env                | no build/connect phases  |
| test push to driver | uvm_sequencer          | + direct mailbox put     |
| mbox                | uvm_sequence           |                          |
| mailbox             | uvm_analysis_port      | TLM-1                    |
| connections         | + uvm_subscriber       |                          |
| aes_config          | uvm_config_db          | handle passed directly   |
| aes_coverage        | uvm_subscriber         | covergroups identical    |
| verbosity int       | 'uvm_info / 'uvm_error | macros wrap \$display    |

The translation is mechanical:

- every run() task becomes the body of a run\_phase task;
- every new() call becomes type\_id::create() once 'uvm\_object\_utils / 'uvm\_component\_utils macros are added to the class declarations;
- every mailbox put/get becomes an analysis-port write or a TLM-1 get\_next\_item/item\_done;
- the aes\_env::connect() method becomes a connect\_phase method.

## APPENDIX D ASSERTION CATALOGUE

TABLE VIII  
BOUND SVA ASSERTIONS (TB/ASSERTIONS/AES\_ASSERTIONS.SV).

| Label               | Behaviour                                                                |
|---------------------|--------------------------------------------------------------------------|
| a_valid_stable      | in_valid && !in_ready   =><br>in_valid && \$stable(data_in)              |
| a_out_stable_under  | !out_valid && !out_ready   =><br>out_valid && \$stable(data_out)         |
| a_bank0_free        | dbg_bank_free[0]   -><br>!dbg_bank_valid[0] &&<br>!dbg_bank_busy[0]      |
| a_bank1_free        | dbg_bank_free[1]   -><br>!dbg_bank_valid[1] &&<br>!dbg_bank_busy[1]      |
| a_aad_hold          | aad_valid && !aad_ready   =><br>aad_valid                                |
| a_deadlock_watchdog | no output progress for 50 000 cycles<br>while inputs arriving => \$fatal |
| a_tag_only_gcm      | tag_valid   -> mode == MODE_GCM<br>(with 4-cycle past window)            |

Additional assertions noted in the plan but implemented as scoreboard checks (because they require sequence-level state that is awkward to express in SVA) include: a\_no\_replay, the a\_fifo\_count\_consistency invariant, and a\_meta\_owner\_pop\_order.

## APPENDIX E COVERAGE REPORT TEMPLATE

The expected coverage summary at end-of-regression takes the form below, generated by the simulator's coverage HTML/text report:

```

=== COVERAGE SUMMARY (aes_coverage) ===
cg_mode : 100.0% ( 36 / 36 bins )
cg_mode_transitions : 97.3% ( 35 / 36 bins )
cg_backpressure : 100.0% ( 30 / 30 bins )
cg_key_banks : 100.0% ( 32 / 32 bins )
cg_dual_bank_swap : 91.7% ( 11 / 12 bins )
cg_gcm : 95.8% ( 23 / 24 bins )
cg_override : 83.3% ( 5 / 6 bins )
cg_serialization : 100.0% ( 4 / 4 bins )
cg_reset_recovery : 100.0% ( 3 / 3 bins )
-----
OVERALL FUNCTIONAL : 96.4%

```

Holes (the missing bins) are documented per Section X and mitigated by directed-test extensions.